user/_echo 文件布局：ELF 头 → 程序头（段）→ 节 → 节头，以及它们之间的指针

按文件偏移从上到下排列（未按比例）· 左侧箭头：头部表指向的位置 · 右侧箭头：节之间的引用 · 最右：段的覆盖范围

LOAD 段　指针段

ELF Header　off 0x000 · 64 B

```
e_ident 7f 'E''L''F' · 64 位 · 小端 · e_type=ET_DYN · AArch64
e_entry  = 0x2c8          入口: main
e_phoff  = 0x40           程序头表: 5 项 × 56 B
e_shoff  = 0xf98 (3992)   节头表: 25 项 × 64 B
```

Program Header Table（段 · 给 exec / ld-xv6）　off 0x040 · 0x118

```
[0] PHDR     off 0x040 → va 0x0040   0x118  R
[1] INTERP   off 0x158 → va 0x0158   0x00f  R
[2] LOAD     off 0x000 → va 0x0000   0x358  R-X
[3] LOAD     off 0x358 → va 0x1358   0x158  RW-
[4] DYNAMIC  off 0x358 → va 0x1358   0x120  RW-
```

.interp　off 0x158 · 0xf

```
"/lib/ld-xv6.so\0"          exec 读它，先装载并跳到解释器
```

.hash (SysV)　off 0x168 · 0x28

```
nbucket = 3      nchain = 5
bucket[] = 1 3 4      chain[] = 0 0 0 2 0
```

.dynsym (5 × 24 B)　off 0x190 · 0x78

```
[0] 空项
[1] name=+0x01 write    FUNC   GLOBAL  UND
[2] name=+0x07 exit     FUNC   GLOBAL  UND
[3] name=+0x0c strlen   FUNC   GLOBAL  UND
[4] name=+0x1b _stack   NOTYPE GLOBAL  0x80000
```

.dynstr　off 0x208 · 0x22

```
\0 write\0 exit\0 strlen\0 libc.so\0 _stack\0
   +0x01   +0x07 +0x0c    +0x13     +0x1b
```

.rela.plt (3 × 24 B)　off 0x230 · 0x48

```
r_offset 0x1498  JUMP_SLOT  sym=1  addend 0
r_offset 0x14a0  JUMP_SLOT  sym=2  addend 0
r_offset 0x14a8  JUMP_SLOT  sym=3  addend 0
```

.plt　off 0x278 · 0x50

```
0x278 PLT0        stp x16,x30 · ldr x17,[0x1490] · br
0x298 write@plt   adrp x16 · ldr x17,[0x1498] · br x17
0x2a8 exit@plt    adrp x16 · ldr x17,[0x14a0] · br x17
0x2b8 strlen@plt  adrp x16 · ldr x17,[0x14a8] · br x17
```

.text　off 0x2c8 · 0x8c

```
0x2c8 main:  stp x29,x30,[sp,#-80]! …
0x2d4 adrp/add x20=0x356 "\n"   x21=0x354 " "
0x304 bl exit@plt          (argc ≤ 1)
0x314 bl strlen@plt
0x324 / 0x344 bl write@plt
```

.rodata　off 0x354 · 4

```
20 00 0a 00    → " \0"  "\n\0"
```

.dynamic (13 × 16 B {tag, val})　off 0x358 → va 0x1358 · 0x120

```
NEEDED   → .dynstr+0x13 "libc.so"
HASH     = 0x168
STRTAB   = 0x208     STRSZ  = 34
SYMTAB   = 0x190     SYMENT = 24
DEBUG = 0 · PLTREL = RELA · FLAGS_1 = PIE · NULL
PLTGOT   = 0x1480
JMPREL   = 0x230     PLTRELSZ = 72
```

.got　off 0x478 → va 0x1478 · 8

```
[0] = 0x1358   _DYNAMIC 的地址
```

.got.plt　off 0x480 → va 0x1480 · 0x30

```
0x1480–0x1490  = 0   保留（惰性绑定用，ld-xv6 不用）
0x1498 = 0x278 (PLT0)   运行时 → libc + write
0x14a0 = 0x278 (PLT0)   运行时 → libc + exit
0x14a8 = 0x278 (PLT0)   运行时 → libc + strlen
```

.symtab .strtab .shstrtab　off 0xa40–0xf95

```
.symtab   0xa40  41 项 × 24（main=0x2c8 …）
.strtab   0xe18  .symtab 的名字
.shstrtab 0xea7  节名字符串（e_shstrndx = 24）
```

Section Header Table（节 · 给 ld / readelf / gdb）　off 0xf98 · 25 × 64

```
[ 1] .interp      PROGBITS  A    0x158
[ 2] .hash        HASH      A    0x168
[ 3] .dynsym      DYNSYM    A    0x190
[ 4] .dynstr      STRTAB    A    0x208
[ 5] .rela.plt    RELA      AI   0x230
[ 6] .plt         PROGBITS  AX   0x278
[ 7] .text        PROGBITS  AX   0x2c8
[ 8] .rodata      PROGBITS  AMS  0x354
[ 9] .dynamic     DYNAMIC   WA   0x1358
[10] .got         PROGBITS  WA   0x1478
[11] .got.plt     PROGBITS  WA   0x1480
[22–24] .symtab .strtab .shstrtab   ([12–21] 调试节略)
```

LOAD #1 • R-X
file 0x000–0x358 → va 0x0

PHDR

INTERP

LOAD #2 • RW-
→ va 0x1358

DYNAMIC

不加载进内存

- ELF 头字段：e_entry / e_phoff / e_shoff
- 节头 → 节（ld、readelf、gdb 用）
- PLT 读 GOT 槽；GOT 初值 0x278 = PLT0
- .dynamic 的 {tag, val} → 各张表（ld-xv6 的目录）
- 程序头 → 段内容（exec、ld-xv6 用）
- 代码引用：bl → PLT，adrp → .rodata（链接时已定）
- 重定位：r_offset → GOT 槽，sym → .dynsym → .dynstr

数据来源：readelf -h -l -S -d -r --dyn-syms、objdump -DD user/_echo · 文件共 5592 字节